Token Usage in the Processminer Skill System

June 12, 2026

CONTENTS

Token Usage in the Processminer Skill System

A Deep-Dive Analysis and Layer-by-Layer Fix Specification

1. Executive Summary

2. What the Measured Data Says

3. The Karpathy LLM-Wiki Pattern as a Token Strategy

Red lines no optimization may cross

Part II — The Fix Specification, Layer by Layer

4. Layer 3 — The Schema Layer: Inject Contracts as Slices, Never Whole

4.1 Current behavior

4.2 Target design

4.3 Why this is principled

4.4 Verification and risks

5. Layer 2 — The Document / View Layer: Project the Wiki, Don't Dump It

5.1 Stop echoing full elements from the writers

5.2 Minify every tool return

5.3 Abridge getProcessElements and getProcessSummary

5.4 Make the document map shared infrastructure (port to the MCP track)

5.5 Verification and risks for the view layer

6. Layer 1 — Raw Sources: Re-Read the Immutable, Don't Carry It

6.1 Current behavior

6.2 Target design

6.3 Verification and risks

7. The Runtime Stratum: Disposable Sessions, Honest Accounting

7.1 Session cycling instead of auto-compaction

7.2 Fix the cost accounting (prerequisite for everything)

7.3 Gemini worker: stable prefix, bounded history

8. The Prompt Layer: Skills That Practice Progressive Disclosure

8.1 Rewrite foundational-run Step 1 (the single largest fix)

Step 1 — Orient from the spine, not the bodies

8.2 Deduplicate the boilerplate into CORE

8.3 Trim the always-loaded surface

8.4 Verification for the prompt layer

9. Rollout Plan

10. What This Plan Deliberately Does Not Do

Token Usage in the Processminer Skill System

A Deep-Dive Analysis and Layer-by-Layer Fix Specification

Date: 2026-06-12 · **Scope:** Both session backends (Claude CLI / MCP and Gemini), the 28 skills, the shared CORE system prompt, and the schema/process-JSON context model · **Constraint:** Every fix adheres to the Karpathy LLM-Wiki pattern and the schema / process-JSON principles.

---

1. Executive Summary

The dominant token cost in Processminer is not what the model writes — it is what gets **replayed**. The runtime store records, for a single dogfood test run, **24.2 million cache-read tokens against just 3,084 fresh input tokens**: the model re-reads an enormous, ever-growing context on every internal model call. Output tokens are outnumbered by context replay roughly 100 : 1.

The cost shape is:

> **(static prompt + hoarded element bodies + accumulated tool results) × number of model calls per session**

Three levers dominate, and all three are not merely compatible with the Karpathy LLM-Wiki pattern — they are *returns to it*:

1. **Skill design** — `foundational-run` (and, in milder form, `document-ingest` and the specialists' orientation phases) loads every element body up front and holds it in the transcript for the whole run. This is both the largest measured cost (7.7 M cache-read tokens in one run) and a violation of the pattern: it promotes the transcript to a shadow source of truth.
2. **Tool-result diet** — the MCP server echoes full elements back on every write, returns full provenance maps on collection reads, and pretty-prints every result (+40–60 %). The stored JSON keeps full fidelity; the view handed to the model does not need it.
3. **Gemini track structure** — the entire 80 KB schema (~20,000 tokens) plus ~26 KB of tool declarations are re-sent on **every** `generateContent` call, and the per-turn document map sits inside the system instruction where it defeats prefix caching.

A fourth, prerequisite finding: **the recorded per-skill dollar costs are inflated roughly 30×** by an accounting bug (the CLI's *cumulative* `total_cost_usd` is summed per turn). The token counts are trustworthy; the dollar figures are not. Fix the measurement first so every other improvement is verifiable.

Estimated impact of the full plan on a typical long skill run: **60–80 % reduction in replayed context**, with the foundational-run redesign alone accounting for the majority.

This document has two parts. **Part I** (sections 2–3) presents the measured findings and the architectural frame. **Part II** (sections 4–9) is the fix specification, organized by layer: the schema layer, the document/view layer, the raw-sources layer, the runtime stratum, and the prompt layer — each with current behavior, target design, step-by-step changes, verification, and risks.

---

2. What the Measured Data Says

Recorded in the runtime store (`data/runtime/dogfood-test-run-2.json`, Claude track, `claude-sonnet-4-6`):

Skill	Output tokens	Cache-read tokens	Cache-creation tokens
foundational-run	59,767	**7,700,601**	204,917
free-chat	60,993	6,617,919	437,257
document-ingest	42,282	6,074,192	125,488
council-review	9,230	1,031,056	15,670
dtp-summary	3,228	989,399	27,587
run-lint	11,127	882,745	38,827
dtp-compare	4,476	928,751	6,153

Reading the table: cache-read tokens are the whole session context replayed on every model call inside every tool round-trip. `foundational-run` ran ~23 minutes interactively; at ~150 K tokens of context per call, ~50 calls explain the 7.7 M replay. The levers are therefore: *shrink what enters the context* and *shorten how long it is carried*.

The accounting bug: the recorded cost for foundational-run is $138.37, but its token counts at Sonnet 4.6 rates compute to roughly $4–5. The Claude CLI's `total_cost_usd` on the `result` event is **cumulative per session**; `src/app/api/session/route.ts` records it every turn and

sums, inflating quadratically. (Fix specified in §7.2.)

---

3. The Karpathy LLM-Wiki Pattern as a Token Strategy

The pattern is not just an audit/trust architecture — it is itself a token strategy. The wiki JSON is the persistent memory, the context window is a disposable workbench, and the schema is the contract between them. Every major waste point found is a place where the implementation strays from the pattern; every fix is a return to it.

LLM-Wiki principle	Token discipline it implies
The wiki JSON is the *only* durable memory	The transcript must never hoard wiki content "for later" — re-read on demand
Context is a derived view of the wiki	Abridged views (document map, schema slice, id + title lists) are computed on read, shown to the model, and discarded — never persisted
Runtime state lives above the wiki (R9)	Session/cursor state in the runtime store makes transcripts disposable — sessions can be cycled freely
Provenance is the trust foundation	Full fidelity lives in the stored JSON only; ephemeral views may omit evidence, the document never does
The schema is the single contract	Schema views are sliced from `schema/process-schema.json` at injection time, never forked into prose

RED LINES NO OPTIMIZATION MAY CROSS

1. Never trim, summarize, or restructure provenance/evidence **in the stored JSON** — only ephemeral views get abridged.
2. Never persist a compacted or abridged representation anywhere (no `document-map.json` sidecar, no cached schema slice on disk). Derived views must be obviously derived and regenerated on every read.
3. Never let a skill instruct the model to treat transcript content as current truth.
4. Session-cycling state goes in the runtime store only — no "resume context" blobs anywhere near `wiki/`.
5. All writes stay behind the schema-validated typed writers. Nothing in this plan changes the mutation path.

Part II — The Fix Specification, Layer by Layer

The Karpathy stack in this codebase has five strata. From the bottom up: **Layer 1** (`raw-sources/`, immutable uploads), **Layer 2** (`wiki/processes/<slug>.json`, the typed document) plus its **view layer** (the read tools that project it into model context), **Layer 3** (the process schema, the contract), the **runtime stratum** above the wiki (session workers, runtime store, accounting), and the **prompt layer** (CORE_SYSTEM_PROMPT + the 28 skills). Each section below fixes one stratum. The layers themselves — the stored JSON, the raw sources, the schema files — are *correct as they are*; what gets fixed is how each layer is **projected into model context**.

---

4. Layer 3 — The Schema Layer: Inject Contracts as Slices, Never Whole

4.1 CURRENT BEHAVIOR

- **Gemini track:** `buildSystemInstruction` (`src/lib/gemini-worker.ts:819–827`) reads `schema/process-schema.json` (80,394 bytes ≈ 20,000 tokens) and concatenates the **entire raw file, with whitespace,** into the system instruction — on every turn, for every skill, regardless of which element types the skill can touch.
- **Claude track:** the schema reaches the model correctly — on demand, via `getSectionContext` (template skeletons per section, ~0.5–1 KB) and the conformance errors of the typed writers. No fix needed there.
- **Prompt layer leakage:** several skills (`transformation-agent` lines ~46–61, `it-architect` ~30–42, `document-ingest` field-mapping passages) restate schema structure in prose — id formats, status defaults, block headings, relation list shapes — ~2.4 KB of forked contract across 6+ skills.

4.2 TARGET DESIGN

One new pure function, one source of truth, two consumers:

```
// src/lib/schema-slice.ts (new)
sliceSchema(schema: ProcessSchema, sections: string[]): SchemaSlice
```

- Input: the parsed custom app schema (already loaded by `getSchema()` / `wiki.ts`) and a list of section names.
- Output: a minified object containing **only** the `elementTypes` whose `section` is in the list, each reduced to what authoring needs: `idPrefix`, `section`, `template` (block headings), `fields` (name, type, required, allowed `fieldValues`), `relations`. Strip descriptions/examples not needed for authoring; strip `fieldValues` entries for types outside the slice.
- A static map `SKILL_SECTIONS: Record<skillName, string[]>` declares which sections each skill owns. It lives next to `SKILL_NAMES` in `gemini-worker.ts` (or a shared module) and mirrors the "Stay in your lane" sections already written in each skill — this is a *routing* table, not a contract fork: every field definition still comes from `schema/process-schema.json` at call time.

Gemini integration: replace the raw-file injection at `gemini-worker.ts:819–827` with:

```
const schema = getSchema();
const sections = SKILL_SECTIONS[activeSkill] ?? ALL_SECTIONS;  // free-chat keeps
everything
instruction += `### Process Schema (sections relevant to this session):\n` +
  JSON.stringify(sliceSchema(schema, sections)) + "\n\n";
```

Expected size: 4–10 KB minified instead of 80 KB raw → **~15,000 tokens saved per `generateContent` call**, which for a 5-call tool loop is ~75 K tokens per turn.

Skill-prose cleanup: delete the restated id/status/blocks passages from the 6 skills and replace each with one sentence: *"The section's exact field structure comes from `getSectionContext` — never restate or assume it."*

4.3 WHY THIS IS PRINCIPLED

The schema remains the single contract; the slice is recomputed from it on every injection and never persisted. The dual-representation rule (custom schema vs. Draft-07 JSON Schema) is untouched — `sliceSchema` reads only the custom schema, exactly like `wiki.ts` and `conformance.ts` already do, and the drift-guard test (`schema-consistency.test.ts`) continues to govern the two files.

4.4 VERIFICATION AND RISKS

- **Test:** a unit test asserting that for every skill in `SKILL_SECTIONS`, the slice contains every element type the skill's tools can write (cross-check against the skill's "owns" list), and that slicing with `ALL_SECTIONS` is information-equivalent to the full `elementTypes` map for authoring fields.

- **Risk:** a skill creating an element type outside its declared sections (e.g. foundational-run mid-run creating `pain-point`, `compliance-gap`, `control`). Mitigation: `SKILL_SECTIONS` lists *writable* sections per skill, not just primary ones — foundational-run's slice includes the As-Is and control sections it can create into. The backend validators are unaffected either way: a wrong write still fails conformance with a precise error, which is the existing contract.
- **Risk:** drift between `SKILL_SECTIONS` and skill prose. Mitigation: the map is small, colocated, and a comment points each entry at the skill's "Stay in your lane" section; a follow-up could generate both from one declaration.

---

5. Layer 2 — The Document / View Layer: Project the Wiki, Don't Dump It

The stored document is sacred and stays byte-identical. Everything in this section changes only the **projection** — what the read/write tools place into model context.

5.1 STOP ECHOING FULL ELEMENTS FROM THE WRITERS

Current (`src/lib/claude-mcp-server.ts`):

```
// createElement, line 764
return { ... text: JSON.stringify({ ok: true, id: built.id, element: built.full-
Element }, null, 2) ... };
// updateElement, line 830
return { ... text: JSON.stringify({ ok: true, id: targetId, element: res.element,
...relationsAdded }, null, 2) ... };
```

The model authored `element` / `patch` one message earlier; echoing it back duplicates 2–5 KB per write into the transcript, replayed on every subsequent call. A document-ingest run with 40 creates parks 80–200 KB of pure duplication.

Target:

```
// createElement
{ ok: true, id: built.id, title: built.fullElement?.content?.title ?? null }
// updateElement
{ ok: true, id: targetId, updatedBlocks: Object.keys(patch?.content ?? {}),
  ...(relationsAdded ? { relationsAdded } : {}) }
```

Keep everything the model could not know: the generated id (it must use it for relations and tempKey resolution), validation warnings, `relationsAdded` from `syncRelations`. Drop everything it just wrote. `createElements` (line 787) already returns counts + ids and needs no change beyond minification.

Skill-side contract check: grep the skills for any instruction that relies on the echo (e.g. "represent the element the tool returns"). foundational-run's `[E]` path re-presents the updated element — it should re-present **from its own redraft**, which it already has in context, or via one `expandElement` if it must show canonical state. Add one sentence to CORE §2 (`updateElement` doc): *"The tool returns a confirmation, not the element; the canonical state is always one `expandElement` away."*

5.2 MINIFY EVERY TOOL RETURN

All ~24 return sites in `claude-mcp-server.ts` use `JSON.stringify(obj, null, 2)` (lines 640, 647, 654, 663, 674, 713, 737, 742, 764, 787, 830, …). Two-space indentation plus per-property newlines inflate every result 40–60 %. **Change:** one helper `const out = (obj: unknown) => ({ content: [{ type: "text", text: JSON.stringify(obj) }] });` and replace every return site. Models parse minified JSON perfectly; nothing else changes. Apply the same to the Gemini tool dispatch where results are stringified before being appended as `functionResponse` parts.

5.3 ABRIDGE `GETPROCESSELEMENTS` AND `GETPROCESSSUMMARY`

Current: `getProcessElements` (`claude-mcp-server.ts:656–663` → `wiki.ts`) returns full `meta` + full `content` per element — including the entire provenance map with verbatim evidence quotes — 3–8 KB per collection where the callers (advisor sessions, orientation phases) need ids, titles and status. Provenance evidence is for the conformance engine and the approval UI; the model reading a collection list never needs it.

Target: default abridged shape per element:

```
{ "id": "PS-COB-001", "title": "Application Receipt", "status": "draft",
  "approval": "open", "confidence": "high", "relations": { "systems": ["SYS-…"],
"controls": [] } }
```

with `detail: "full"` as an explicit opt-in parameter for the rare caller that genuinely needs bodies. `getProcessSummary` (line 649–654) similarly returns `doc.process.meta` wholesale as `overview`; reduce to `{ id, status, owner, docStatus }` + the description.

Provenance stays reachable, not ambient: `expandElement({ type, id })` continues to return the complete element including provenance — that is the *deliberate* full read, used exactly when an element is being worked. The change removes provenance from *ambient* collection scans only. Stored JSON: untouched.

5.4 MAKE THE DOCUMENT MAP SHARED INFRASTRUCTURE (PORT TO THE MCP TRACK)

The Gemini track's `generateDocumentMap` (`gemini-worker.ts:675–718`) is the progressive-disclosure model done right: root `meta` / `content` preserved, the active element full-expanded, peers as `{id, title}`, schema-driven referenced collections as `{id, title}`, everything else as a count. Measured compression: a 150 KB process → ~1.5 KB map (99.5 %). The MCP track has no equivalent — its sessions fall back to `getProcessSummary` (coarse) or repeated full expands (expensive).

Target:

1. Move `generateDocumentMap` + `customStringify` out of `gemini-worker.ts` into `src/lib/document-map.ts` (pure functions over the parsed doc — no I/O).
2. Fix the hard-coded `schemaReferences` map (`process-step → roles, systems`, etc., lines 677–681): derive it from each element type's `relations` declaration in the schema, so the map follows the contract instead of a stale literal.
3. Expose it on the MCP server as `getDocumentMap({ slug, activeType?, activeId? })`, returning the map minified. Register it in the tool list with a 2-line description.
4. Reference it from CORE §1 so both tracks describe the same context model: the Document Map is *fetched*, cheap, and always current — re-fetch it rather than reconstruct state from memory.

5.5 VERIFICATION AND RISKS FOR THE VIEW LAYER

- **Verification:** `npm run typecheck`; `npm test`; a `/dogfood-run` pass comparing per-skill `cacheReadTokens` before/after (the runtime store gives this for free once §7.2 lands); spot-check one full skill flow per track to confirm no skill step depended on a removed echo.
- **Risk:** a model occasionally wanting the post-write canonical element (e.g. to verify back-end-applied defaults). Mitigation: that is one `expandElement` away, and the writers already return validation errors when something was rejected — silence means applied-as-sent.
- **Risk:** abridged `getProcessElements` starving a consumer that silently relied on bodies (advisor preambles, area-summary). Mitigation: audit each caller before flipping the default; `detail: "full"` is the escape hatch and the change is per-tool, not global.

---

6. Layer 1 — Raw Sources: Re-Read the Immutable, Don't Carry It

6.1 CURRENT BEHAVIOR

`document-ingest` (6.07 M cache-reads measured) reads the uploaded document end-to-end (SKILL.md step 1), then extracts dozens of elements across many tool round-trips — with the full document text *and* every extraction riding in the transcript for the whole run. `dtp-regenerate` / `dtp-compare` similarly hold the original DTP plus the As-Is.

6.2 TARGET DESIGN

Layer 1 is immutable by contract — which makes re-reading it always safe and always current. Restructure the ingest prompt to a **two-pass, per-section flow**:

1. **Pass 1 — outline only.** Read the document once; produce the outline/summary the SME confirms (this already exists in the skill). From the outline, build a section→pages/regions work-list. The work-list is small; it can live in the transcript safely.
2. **Pass 2 — per-section extraction.** For each work-list entry: re-read *only that region* of the source (Claude Code's `Read` supports offset/limit; PDFs support page ranges), extract that section's elements via `createElements` (batched), verify the drafts against the same region, then move on. The conflict log goes where it already goes — the ingest report / conflicts store — not the transcript.
3. The skill text changes from "Read the document end-to-end [and work from it]" to "Read the document end-to-end **once for the outline**; during extraction, re-read each section's pages as you reach them — the source is immutable, a re-read is always correct, and you must never quote it from conversational memory."

That last clause is the provenance win as well as the token win: `document` provenance requires **exact quotes**, and an exact quote re-read from the file beats one recalled from a 40-turn-old transcript copy.

6.3 VERIFICATION AND RISKS

- **Verification:** ingest the same source document before/after; diff the resulting process JSON (element count, provenance evidence quotes) — they should be equivalent or better (fewer paraphrased "quotes").
- **Risk:** cross-section content (a control mentioned in two chapters) being missed by regional reads. Mitigation: pass 1's outline explicitly tags cross-references; pass 2 may re-read any region at will — re-reads are cheap and always allowed, the rule is only against *carrying*.

---

7. The Runtime Stratum: Disposable Sessions, Honest Accounting

This stratum sits above the wiki (R9) — `session-worker.ts`, the runtime store, the route. It is where transcript lifetime and measurement are governed.

7.1 SESSION CYCLING INSTEAD OF AUTO-COMPACTION

Problem: a long foundational run accumulates context until the Claude CLI auto-compacts. Compaction produces an *unprovenanced paraphrase* of wiki content the model then works from — a shadow wiki in the transcript, plus a large one-off summarization cost.

Design — cycle on the cursor, not on compaction:

1. The route already tallies per-turn usage (`extractUsage`). Track per session: `cacheCreationTokens` cumulative and last-turn `cacheReadTokens` (a direct proxy for context length).
2. When a turn's `cacheReadTokens` crosses a threshold (e.g. 120 K) **and** the active skill is resumable (`foundational-run`, `qer-session` — exactly the skills with a runtime cursor), the route ends the worker after the turn completes: `worker.dispose()`, drop the `sessionId` binding.
3. The next user message arrives with no live session → the pool spawns a fresh worker; the wire message carries the same scope preamble + skill handle; the skill's own Step 2 ("State exists, not done → resume from `current`") does the rest. **No new state is invented** — the runtime cursor already makes the run resumable across restarts; cycling merely uses that on purpose.
4. UX: the reply stream notes "(session refreshed — resuming at item N of M)" so the SME understands the seam. The skill already mandates exact resume wording via `getSessionStatus`'s fixed strings, so the seam is wording-stable.
5. Non-resumable skills (one-shot button skills like `area-summary`, `dtp-*`) never trigger cycling — they end anyway.

Why principled: the transcript is disposable *because* every durable fact lives in the wiki and every orchestration fact lives in the runtime store. Cycling is the architecture exercising its own guarantee; auto-compaction is the harness papering over a violation of it.

Risks: conversational nuance (the SME's tone, a deferred aside) does not survive the seam. Mitigation: cycle only at item boundaries (after an outcome lands, before the next `advanceSession`), never mid-element; threshold high enough that cycling is rare (once or twice per long run).

7.2 FIX THE COST ACCOUNTING (PREREQUISITE FOR EVERYTHING)

Problem: `extractUsage` (`src/lib/token-usage.ts`) reads the CLI result event's `total_cost_usd` — **cumulative per session** — and the route (`route.ts` ~284) records it every turn into `skillUsage`, summing. With ~50 turns ramping linearly, the recorded sum is ~25× the true final cost ($138 recorded vs ~$4–5 computed for foundational-run).

Design:

1. In the route's result handler, keep a per-`sessionId` record of the last seen cumulative value: `prevCost`, and (after empirically confirming whether the `usage` token fields are per-turn or cumulative — log two consecutive turns and compare) `prevTokens` if needed.
2. Record `delta = max(0, current − prev)` per turn; update `prev`. Persist `prev` in the session map (`getSessionInfo` storage) so a rehydrated worker (`--resume` after server restart) doesn't double-count its first turn.
3. Gemini's `usageMetadata` is already per-turn (summed across the turn's calls in `turnUsage`) — no change.
4. Add a regression test: feed two synthetic result events with cumulative costs 1.0 then 2.5; assert recorded totals are 1.0 + 1.5.
5. Optional but cheap: surface `cacheReadTokens` per turn in the session debug log — it is the context-length gauge that §7.1's threshold and all before/after comparisons read.

Usage tallies remain in the runtime store; the wiki never sees them.

7.3 GEMINI WORKER: STABLE PREFIX, BOUNDED HISTORY

Three changes in `gemini-worker.ts`, in order of leverage:

1. **Move the Document Map out of `systemInstruction`** (built at line 898, injected at 981 on every call). It is the only per-turn dynamic part of the instruction; at the *front* of the request it invalidates Gemini's implicit prefix cache for the entire instruction + tool declarations behind it. Inject it instead as the leading part of the latest user turn (`history.push` site, line 947): `[Current Document Map — regenerated this turn]\n…\n\n[SME message]\n…`. The system instruction becomes **stable for the session** (CORE + schema slice + skill), which is exactly what implicit caching rewards. Explicit `cachedContent` for that stable prefix is the follow-up step once this lands.
2. **Schema slice** per §4.2 (~15 K tokens/call).

3. **Bound the loop's accumulation.** `history` grows unboundedly (push sites 947, 1513–1514; cleared only on `dispose`, line 1603) and tool responses are appended verbatim with no size cap (lines 1491–1497). Add: (a) a per-result cap (e.g. 4 K chars, with a `…truncated – call expandElement for the full element` tail) — safe because every truncated result is a *view*, recoverable from the wiki by tool call; (b) the same cycle-on-threshold logic as §7.1, using `promptTokenCount` as the gauge, since the GeminiWorker also rehydrates from the session map. Do **not** silently window `history` mid-session — dropped turns break tool-call/response pairing; cycling at turn boundaries is the safe equivalent.
4. **Pin the active skill per session.** The keyword scan (lines 829–859) can flip `activeSkill` mid-session when an SME message happens to contain another skill's name, swapping a 3–20 KB block inside the instruction and busting the cache. Once set (and persisted via `saveSessionSlug`), only an explicit `skill` parameter on `runTurn` should change it.

---

8. The Prompt Layer: Skills That Practice Progressive Disclosure

8.1 REWRITE FOUNDATIONAL-RUN STEP 1 (THE SINGLE LARGEST FIX)

Current text (`.claude/skills/foundational-run/SKILL.md:48–57`): take the snapshot, then *"read **every current-state element body** once (`expandElement({ type })` … then `expandElement({ type, id })` for the bodies) and **keep those bodies as your working copy for the whole run** — do not re-`expandElement` an unchanged element later."*

For a 150 KB process this parks ~37 K tokens in the transcript from turn 1 and replays them on every one of ~50+ calls (~1.85 M token-reads), versus ~1.5 K per item paid once under lazy expansion. Worse than the cost: it is a standing instruction to trust stale data — the SME can edit an element in the app mid-run, `applyLint` can rewrite one, and the transcript copy silently diverges from the wiki.

Replacement Step 1 (drop-in):

> ## Step 1 — Orient from the spine, not the bodies
>
> Take the snapshot: `getProcessSummary({ slug })` **once** — per-section counts, section status, the overview. Then `getProcessRelations({ slug })` **once** — the per-step map of linked roles, systems, controls and touchpoints, plus the orphans and uncovered steps. Together these are your whole-process picture: the spine, every relation, what is thin, what does not connect. **Do not expand element bodies during orientation** — you will read each element in full at the moment you challenge it (Step 3), and that read is always the current truth. The wiki is the working copy; your context is not. If a challenge needs a related element's body (an exception that contradicts a step's output), expand *that* element *then* — re-reads are cheap and always correct; carried copies go stale the moment the SME or a lint pass edits the wiki.
>
> You do **not** need to work out which steps lack a control — `uncoveredSteps` and `currentHasControl` report that deterministically (Step 2). Then give the SME the one-line orientation: *"I've read **{process}** — {n} steps, {n} controls, {n} exceptions. The spine is coherent; a few steps read thin; {n} steps carry no control yet. Starting the foundational run."*

Step 3 add-on (one sentence at the top): *"First `expandElement({ type, id })` the `current` element — present from that read, never from memory of an earlier turn."* The rest of Step 3 (challenge shape, probes, outcomes, `[Y]` / `[E]` mechanics) is unchanged.

Expected effect: per-call context drops by the full body-hoard (~37 K tokens for a 150 KB process) for the entire run; element bodies enter the transcript once each, adjacent to where they are used; elements only batch-presented (the gap tail) never need full bodies at all. On the measured run this is the majority of the 7.7 M replay.

Same medicine, smaller doses:

- `document-ingest`: §6.2's per-section flow.
- The six perspective specialists' "Phase 1 — Orientation": replace "read the documented process steps" with the summary + relations orientation; expand a body only when an element is being discussed.
- `qer-session`: it orchestrates the specialists, so it inherits the fix; additionally it is resumable (QER cursor) and therefore in §7.1's cycling set.

8.2 DEDUPLICATE THE BOILERPLATE INTO CORE

~30 KB of near-identical text across the 28 skills, all already governed by `CORE_SYSTEM_PROMPT.md`:

1. **Y/E/R restatements** (8 skills, ~380 B each): replace each with *"Follow the universal Y/E/R loop and batching rules (CORE §3)"* plus only the skill-specific deltas (e.g. foundational-run's four-outcome variant, which genuinely differs and stays).
2. **WEB-PROVENANCE-BLOCK** (3 × 1,367 B, hand-synced by comment): move verbatim into CORE §4 as "Web-sourced provenance"; the three `source-*` skills reference it. The hand-sync comment disappears.
3. **Close-out templates** (10 skills, 5 wording variants): the backend already owns fixed wording elsewhere (`outcomes_line`, `closeout_template` from `getSessionStatus`) — extend the same approach: the close-out text comes from the tool, the skill relays it character-for-character. One canonical source, zero drift.
4. **"Stay in your lane"** (8 skills, ~350 B each): keep — it is genuinely per-skill — but compress to the two lists (owns / doesn't own) plus the one-line hand-off idiom, dropping the repeated rationale paragraphs.

8.3 TRIM THE ALWAYS-LOADED SURFACE

1. **Frontmatter descriptions:** all 28 ship in every Claude-track session (~14.4 KB ≈ 3,600 tokens). Rewrite each to ≤ 250 chars: *what it does + when to route to it*. The long "even if they don't say X" trigger lists compress to 3–4 keywords. Target ≈ 7 KB total (~1,800 tokens saved per session, every turn).
2. **Evict the dogfood harnesses:** `dogfood-run` / `dogfood-target` / `dogfood-dtp` (50 KB bodies + 1.9 KB descriptions) move from `.claude/skills/` to `dogfood/skills/`, wired in only for test sessions (e.g. a `--add-dir`/settings include the harness run uses, or a symlink the dogfood instructions create and remove). Production SME sessions never load them.
3. **CLAUDE.md split (optional, trade-off):** ~9.6 KB of developer documentation loads into every SME session worker. If pursued: keep the session-relevant invariants (never hand-edit the JSON, tool layer, provenance pointer) in CLAUDE.md and move the architecture narrative to `TARGET-ARCHITECTURE.md` links. Defer until after the bigger levers land — it trades against developer ergonomics.

8.4 VERIFICATION FOR THE PROMPT LAYER

The dogfood harnesses are the regression suite here: run `/dogfood-run` (and `/dogfood-target` for the to-be track) before and after each skill rewrite; assert behavior parity on every interaction path (Y / E / R / Deep Dive / Move On) and compare per-skill `cacheReadTokens` and wall-clock from the runtime store. The foundational-run rewrite specifically must show: same challenge quality (spot-check transcripts), same approvals written, materially lower cache-read totals.

---

9. Rollout Plan

Phase	Items	Layer	Effort	Risk	Gate
1	§7.2 cost accounting fix	Runtime	S	Low	Unit test; recorded ≈ computed cost on one live run
2	§5.1 write-echo removal · §5.2 minify · §5.3 abridged reads	View	S	Low	typecheck + tests + one dogfood pass per track
3	§8.1 foundational-run rewrite (+ specialists' Phase 1, §6.2 ingest)	Prompt + L1	M	Med	/dogfood-run parity + cache-read delta
4	§4.2 schema slice · §7.3 Gemini prefix/history	L3 + Runtime	M	Med	Gemini turn-token logs before/after; cache-hit rate
5	§8.2 boilerplate dedup · §8.3 descriptions + dogfood eviction	Prompt	S	Low	routing spot-checks (skill triggers still fire)
6	§7.1 session cycling · §5.4 shared getDocumentMap	Runtime + View	M	Med	long-run dogfood crossing the threshold; resume seam check

Expected cumulative effect on a long skill run: **60–80 % less replayed context**, with phase 3 the dominant contributor on the Claude track and phase 4 on the Gemini track. Every phase is independently shippable and independently measurable once phase 1 lands.

---

10. What This Plan Deliberately Does *Not* Do

To stay inside the schema and process-JSON principles:

- No trimming of provenance or evidence in the stored document — traceability is the product.
- No abridged copies persisted anywhere; document maps and schema slices are computed on read, and even then never written into `wiki/`.
- No bypassing of the typed writers; the mutation path is untouched.
- No merging of the two schema representations (custom app schema vs. Draft-07 JSON Schema) — the drift-guard test continues to govern them.

- No history-windowing inside a live session (it breaks tool-call pairing and creates silent amnesia) — session cycling at item boundaries, with the runtime cursor, is the principled substitute.

Prepared by Claude Code · Sources: `src/lib/claude-mcp-server.ts`, `src/lib/gemini-worker.ts`, `src/lib/session-worker.ts`, `src/lib/token-usage.ts`, `src/lib/wiki.ts`, `src/app/api/session/route.ts`, `.claude/skills/` *(28 skills + CORE_SYSTEM_PROMPT.md),* `schema/process-schema.json`, `data/runtime/*.json` *recorded usage.*